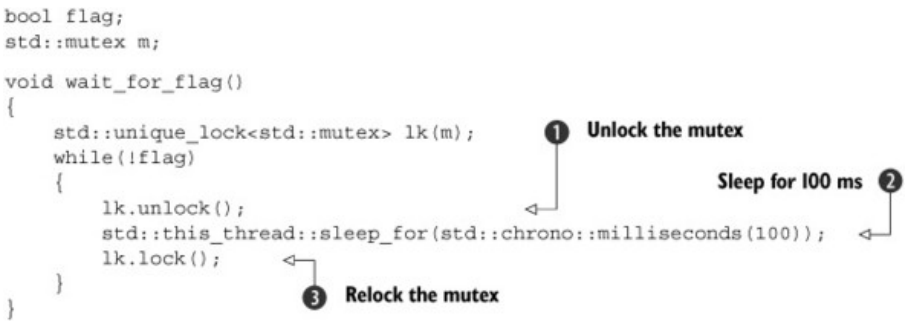
A second option is to have the waiting thread sleep for small periods between the checks using the `std::this_thread::sleep_for()` function (see section 4.3):



In the loop, the function unlocks the mutex ❶ before the sleep ❷ and locks it again afterward ❸, so another thread gets a chance to acquire it and set the flag.

This is an improvement, because the thread doesn't waste processing time while it's sleeping, but it's hard to get the sleep period right. Too short a sleep in between checks and the thread still wastes processing time checking; too long a sleep and the thread will keep on sleeping even when the task it's waiting for is complete, introducing a delay. It's rare that this oversleeping will have a direct impact on the operation of the program, but it could mean dropped frames in a fast-paced game or overrunning a time slice in a real-time application.

The third, and preferred, option is to use the facilities from the C++ Standard Library to wait for the event itself. The most basic mechanism for waiting for an event to be triggered by another thread (such as the presence of additional work in the pipeline mentioned previously) is the *condition variable*. Conceptually, a condition variable is associated with some event or other *condition*, and one or more threads can *wait* for that condition to be satisfied. When some thread has determined that the condition is satisfied, it can then *notify* one or more of the threads waiting on the condition variable, in order to wake them up and allow them to continue processing.

4.1.1. Waiting for a condition with condition variables

The Standard C++ Library provides not one but *two* implementations of a condition variable: `std::condition_variable` and `std::condition_variable_any`. Both of these are declared in the `<condition_variable>` library header. In both cases, they need to work with a mutex in order to provide appropriate synchronization; the former is limited to working with `std::mutex`, whereas the latter can work with anything that meets some minimal criteria for being mutex-like, hence the `_any` suffix. Because `std::condition_variable_any` is more general, there's the potential for additional costs in terms of size, performance, or operating system resources, so `std::condition_variable` should be preferred unless the additional flexibility is required.

So, how do you use a `std::condition_variable` to handle the example in the introduction —how do you let the thread that's waiting for work sleep until there's data to process? The following listing shows one way you could do this with a condition variable.

Listing 4.1. Waiting for data to process with a `std::condition_variable`